

# From No-Injection to Exactness in Reusable Committed-Set PSI

Anonymous Authors

Anonymous Institution

**Abstract.** Committed-token PSI and reusable committed-set PSI can often enforce *no-injection*: a malicious party may be forced to use only elements from a committed hidden set. No-injection is not exactness, because the party may still run a later session on a strict subset. We give a compiler that upgrades compatible no-injection committed-token PSI to exact reusable-set consistency. The compiler combines same-element binding with an *owner-hidden digest*: raw digest tags remain hidden from the state owner, and an accepted strict omission requires a nonempty hidden tag sum to be zero. The adaptive failure term is bounded by  $q_{\text{dig}}/(|\mathbb{F}|^\ell - q_{\text{dig}})$  plus standard component losses.

We formalize the compatible committed-token PSI interface needed by the compiler, define an owner-hidden digest abstraction, and give two realizations. The generic realization evaluates digest tags inside malicious-secure arithmetic 2PC. The cached authenticated ShareTag realization moves PRF evaluation to setup/update time: each inserted element receives an authenticated owner share and an encrypted verifier-share handle, while sessions perform only aggregate binding and aggregate-decryption equality checks. We also give a static bilinear accumulator backend witnessing realizability of the interface and a dynamic hidden-state application, HDUA-LAUSE, where a bilinear set-polynomial accumulator supplies update validity and session no-injection. A VA-UPSI corollary illustrates composition with a flagged committed-token PSI backend. The construction is an exact-state authentication layer, not a faster live-query PSI engine: the PSI backend still processes active records, while exactness no longer relies on complete historical handle enumeration.

## 1 Introduction

Private set intersection protocols are usually analyzed per execution. In reusable-input and dynamic settings, however, the same party may participate in many sessions under a claimed committed state. A natural commit-and-prove relation can prevent *injection*: every active PSI record must open to an element of the committed set. This does not imply exact use. A malicious committer may still omit committed elements and run the PSI session on a strict subset.

As a concrete example, suppose a party commits to  $S = \{a, b, c\}$  and later runs PSI using only  $\{a, c\}$ . A no-injection check accepts this session, because every active record belongs to the committed set. Nevertheless, the session is not exact: the committed element  $b$  has been silently omitted. The compiler in

this paper rules out this subset attack without requiring the verifier to learn the committed elements.

This paper studies the gap between no-injection and exact use. The central observation is that no-injection becomes exactness if it is paired with an owner-hidden verifier-keyed digest. For a hidden set  $S$ , let

$$A(S) = \sum_{x \in S} \Gamma_{K_{\text{dig}}}(x) \in \mathbb{F}^\ell.$$

If a session relation proves  $S_{\text{use}} \subseteq S$  and a private digest check proves  $A(S_{\text{use}}) = A(S)$ , then any strict omission requires the nonempty omitted set to have zero hidden digest. When the state owner never learns raw tags or linear information beyond the final equality bit, this occurs with probability at most  $q_{\text{dig}}/(|\mathbb{F}|^\ell - q_{\text{dig}})$  in the adaptive rejection model, and hence  $q_{\text{dig}}/|\mathbb{F}|^\ell + \text{negl}(\lambda)$  for polynomially many digest attempts and superpolynomial tag space.

*Contributions.* Our first contribution is an Exact-PICS compiler for compatible committed-token PSI interfaces. The interface is intentionally not black-box for arbitrary PSI: it must expose extractable active records, no-injection, active-record distinctness, same-element binding across tokens, commitments, representatives and digest inputs, and token privacy. Under these conditions, the hidden digest protocol upgrades subset consistency to exact reusable-set consistency.

Our second contribution is an owner-hidden digest interface and two realizations. The generic realization  $\Pi_{\text{HDig}}^{2\text{PC}}$  uses a stable digest domain independent of session identifiers and epoch handles, committed key shares, committed additive digest shares, and malicious-secure arithmetic 2PC for setup, update, and session checks. The optimized cached tag-share realization  $\Pi_{\text{HDig}}^{\text{ShareTag}}$  generates owner shares together with encrypted verifier shares when elements enter the state; later sessions aggregate owner shares and encrypted verifier shares and run only an aggregate-decryption equality check. Public homomorphic certified-tag handles are treated only as a leakage-explicit deployment variant, not as the hidden-state theorem.

Our third contribution is a dynamic application, digest-native HDUA-LAUSE. A hidden bilinear set-polynomial accumulator proves update validity and session no-injection. A hybrid proof model is used: the arithmetic commit-and-prove layer is knowledge-extractable, while a Groth-Sahai-style pairing layer supplies soundness for linked pairing-product equations. Unique-state binding is reduced to commitment binding, representative collision resistance, and a bounded polynomial evaluation binding assumption for the structured reference string.

The first two contributions are the conceptual core. The cached authenticated tag-share realization is a practical optimization of the same owner-hidden digest functionality, not a separate assumption. The dynamic HDUA-LAUSE construction and the VA-UPSI composition are applications showing that once a state-authentication layer supplies no-injection for the current hidden state, the same hidden digest compiler upgrades no-injection to exact current-state use.

*Scope.* The construction authenticates exact hidden state use. It is not a sublinear live-query PSI protocol. The PSI backend still evaluates active records. The gain is that exactness is obtained from no-injection plus hidden digest equality, rather than from complete enumeration of every historical base/delete/add handle. Local rollback and fork resistance require a stateful verifier; global non-equivocation requires an external transparency log or append-only service.

*Roadmap.* Section 2 gives the compiler intuition and invariant chain. Section 4 defines the compatible no-injection committed-token PSI interface. Section 5 defines owner-hidden digests, while Sections 6 and 7 instantiate them by generic 2PC and cached ShareTag. Section 8 gives the Exact-PICS compiler, the main theorem of the paper. Sections 9–11 show realizability through bilinear accumulators and a dynamic hidden-state application. Section 12 states the layered security results. Section 13 records the VA-UPSI composition as a corollary/application. Section 14 gives efficiency and limitations.

## 2 Technical Overview

### Central exactness principle

Let  $S$  be the hidden set extracted from an accepted commitment or epoch state. Suppose a session relation proves  $S_{\text{use}} \subseteq S$ , proves that active records are distinct, and runs a verifier-keyed hidden digest check showing

$$\sum_{x \in S_{\text{use}}} \Gamma_{K_{\text{dig}}}(x) = \sum_{x \in S} \Gamma_{K_{\text{dig}}}(x).$$

If the malicious state owner does not learn raw digest tags or an oracle that enables adaptive linear search beyond the final digest-check accept/reject bit, then  $S_{\text{use}} = S$  except with probability at most  $q_{\text{dig}}/(|\mathbb{F}|^\ell - q_{\text{dig}})$  plus PRF, proof, binding, and digest-protocol losses.

Returning to the example  $S = \{a, b, c\}$ , no-injection allows a malicious session on  $S_{\text{use}} = \{a, c\}$ . The digest check compares

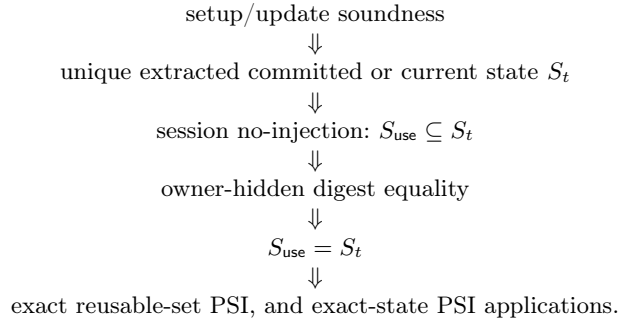
$$\Gamma_K(a) + \Gamma_K(c) \quad \text{with} \quad \Gamma_K(a) + \Gamma_K(b) + \Gamma_K(c).$$

The check can pass only if  $\Gamma_K(b) = 0$ . Since  $K$  is hidden from the state owner and the tag is never opened, the state owner cannot search for such an omission except through the final equality test. The formal proof handles adaptive rejected tests and gives the  $q_{\text{dig}}/(|\mathbb{F}|^\ell - q_{\text{dig}})$  term.

The compiler has three layers. First, a compatible committed-token PSI relation provides no-injection and same-element binding. Second, an owner-hidden digest realization maintains a hidden algebraic digest of the committed state. The generic 2PC realization stores committed additive shares and checks equality inside 2PC; the cached tag-share realization stores owner shares and

encrypted verifier-share handles for each tag and checks equality by aggregate share recovery. Third, the session relation checks that the active records have the same hidden digest as the certified state. In the dynamic construction, a hidden accumulator supplies the no-injection and update-validity relation; the same digest principle supplies no-omission.

The components have separate roles. The base committed-token relation prevents injection. Same-element binding prevents the prover from using one element for the PSI token and another for the digest. The hidden digest prevents omission. The accumulator used later is not the source of no-omission; it is the dynamic mechanism that supplies update validity and no-injection.



### 3 Related Work and Positioning

The paper sits at the intersection of committed/reusable PSI, malicious PSI, authenticated set representations, and dynamic PSI. The contribution should not be read as a new random fingerprint or a new fast PSI engine. The contribution is a compiler that makes a fingerprint usable in a hostile committed-token setting: the digest is verifier-keyed, hidden from the state owner, bound to the same hidden records as the PSI tokens, and analyzed under adaptive rejected equality tests.

*Committed and reusable PSI inputs.* Modern PSI protocols based on OPRFs, VOLE, OKVS/PaXoS encodings, and circuit PSI provide strong guarantees for a single execution [16,17,9]. These guarantees do not by themselves force a party to reuse the same input across sessions. PICS studies private intersection over committed reusable sets and explicitly targets cross-session input consistency [10]. The Exact-PICS compiler targets the specific residual gap between subset/no-injection consistency and exact use: if a base relation proves only that active records form a subset of the committed set, the owner-hidden digest check rules out strict subsets.

*Verifiable PSI and commit-and-prove structure.* Malicious or verifiable PSI typically protects a single protocol execution. Our compiler requires a stronger commit-and-prove interface: the active records consumed by the PSI backend

must be extractable, distinct, and linked to the same hidden elements used by the digest and membership relations. This is why the result is not a black-box transformation of arbitrary PSI. It applies to committed-token PSI protocols that expose the right active-record and same-element-binding structure.

*Accumulators, vector commitments, and authenticated data structures.* Authenticated set representations provide membership, nonmembership, and update evidence. RSA and bilinear accumulators, Merkle-style authenticated dictionaries, and vector commitments all provide ways to bind a public handle to a hidden or committed collection [4,15,14,6,7]. Our static bilinear backend is used only to witness realizability of the no-injection interface. The dynamic HDUA-LAUSE application uses a bilinear set-polynomial accumulator because it composes naturally with batch update equations and pairing-product proofs. The accumulator is not responsible for no-omission; it provides update validity and no-injection, and the owner-hidden digest upgrades that no-injection guarantee to exactness.

*Set-membership and proof systems.* Zero-knowledge set-membership and accumulator proofs are well studied [8,5]. We use a hybrid proof-system model: arithmetic commit-and-prove extracts scalar witnesses such as representatives, coefficients, shares, and openings, while a Groth-Sahai-style layer gives soundness for linked pairing-product equations [11]. The main text states the concrete relation formats; the appendix gives the reductions.

*UPSI and dynamic computation.* UPSI protocols reduce the cost of repeated intersections over evolving sets. The original UPSI line and UPSI Revisited study update-dependent costs, extended functionalities, arbitrary deletions, and worst-case update-size complexity in their chosen security models [2,3]. Structured-encryption UPSI gives another approach for dynamic insertions and deletions [1]. The present work is orthogonal: it supplies malicious exact-state authentication for hidden evolving sets, not a replacement for high-performance dynamic PSI engines.

*Random fingerprints and algebraic equality tests.* Random linear fingerprints and polynomial identity tests are classical tools for checking equality with small error. Used naively, however, they do not solve reusable committed PSI: the malicious state owner might choose the digest key, learn raw tags, use different elements for the digest and PSI token, or adaptively test many rejected subsets. The digest layer in this paper addresses exactly those issues by making tags owner-hidden, same-element-bound, and adaptively accounted.

In summary, prior PSI work gives efficient single-session security, PICS-style work gives reusable committed inputs and no-injection or consistency mechanisms, and UPSI gives update-efficient dynamic computation in its security model. Our contribution is orthogonal to all three: we give a compiler that upgrades no-injection to exact use while preserving hidden inputs. The dynamic accumulator construction shows how the same compiler principle can be applied to evolving hidden states.

## 4 Base Interface: Compatible No-Injection Committed-Token PSI

The Exact-PICS compiler is not black-box for arbitrary PSI. It applies to a base protocol that exposes enough structure to bind the PSI token, committed element, accumulator representative, and digest input.

The reason is simple: the compiler must know, in a proof-theoretic sense, which hidden records were actually supplied to the PSI backend. It must also know that these records are the same hidden elements used for digest evaluation. Without these requirements, a prover could pass the digest check for one set of committed elements while feeding a different set of tokens into PSI.

**Definition 1 (Compatible no-injection committed-token PSI).** *A compatible no-injection committed-token PSI interface, written NI-ctok-PICS, consists of setup and session protocols with the following properties.*

1. **Setup extractability.** *From an accepting malicious setup transcript, an extractor obtains a canonical committed set  $S = \{x_1, \dots, x_n\}$ , element commitments  $C_i = \text{Com}(x_i; r_i)$ , and openings sufficient to link records to elements.*
2. **Session active-record extraction.** *From an accepting malicious session proof, an extractor obtains the effective active set  $S_{\text{use}}$  represented by the PSI records consumed by the backend.*
3. **No-injection.** *Except with soundness error, every accepting session satisfies  $S_{\text{use}} \subseteq S$ .*
4. **Distinctness.** *Active records are a set, not a multiset. Equivalently, extracted representatives are pairwise distinct.*
5. **Same-element binding.** *For every active record, the token input, element commitment input, digest input, and accumulator or membership input are the same hidden element.*
6. **Token privacy.** *The token layer reveals only the authorized PSI output and the leakage explicitly allowed by the functionality.*

*Compatibility map.* The interface is strong, but each requirement has a concrete role: extraction identifies the committed and active sets, no-injection gives the subset relation, distinctness prevents multiset compensation, and same-element binding connects the PSI token to the digest input. The static bilinear backend used in Section 9 witnesses the full interface. PICS-style systems can satisfy the interface when their commit-and-prove layer exposes compatible active-record proofs; arbitrary PSI transcripts do not.

| Requirement          | Static backend    | PICS-style protocols     | Arbitrary PSI |
|----------------------|-------------------|--------------------------|---------------|
| Setup extraction     | ✓                 | committed setup          | no            |
| Active extraction    | ✓                 | session proofs           | no            |
| No-injection         | membership proof  | subset proof, if exposed | no            |
| Distinctness         | arithmetic proof  | extra proof              | no            |
| Same-element binding | link relation     | compatible tokens        | no            |
| Token privacy        | backend-dependent | inherited                | ✓             |

**Same-element binding relation  $R_{\text{SEB}}$** 

The public statement contains a session identifier  $\text{sid}$ , an epoch handle  $h$ , an element commitment handle  $C_x$ , a token commitment or hidden token handle  $C_{\text{tok}}$ , an accumulator commitment  $C^{\text{Acc}}$  when applicable, and the digest public handle for the chosen owner-hidden digest realization. The witness contains  $x$  and the relevant openings or membership witnesses. The relation checks:

$$C_x = \text{Com}(x; r_x), \quad C_{\text{tok}} = \text{Com}(\text{Tok}(\text{sid}, x); r_{\text{tok}}),$$

that  $a_x = \text{H}_{\text{elt}}(x)$  is the representative used in the accumulator relation, and that the same  $x$  is the input wired into the hidden digest protocol. This relation is the non-black-box compatibility point required by the compiler.

The first three properties give the subset relation  $S_{\text{use}} \subseteq S$ . Distinctness rules out multiset attacks in which a prover compensates for an omission by repeating another record. Same-element binding is the glue between the PSI and digest worlds. Token privacy is inherited from the underlying PSI backend and ensures that the compiler does not reveal extra nonintersection information.

The restriction is deliberate. It prevents an overclaim that the compiler upgrades any PSI protocol. The compiler upgrades only protocols whose commit-and-prove/session-token layer exposes extractable, same-element-bindable active records. The relation  $R_{\text{SEB}}$  should be read as a wiring constraint: it does not by itself prove membership or digest equality, but it guarantees that all later checks refer to the same hidden value  $x$ .

## 5 Owner-Hidden Digest Interface

The compiler uses the digest layer only through an abstract interface. This interface is intentionally separated from its realizations. The generic 2PC construction realizes it by evaluating tags inside malicious-secure 2PC. The cached tag-share construction realizes it by preprocessing authenticated additive tag shares. In both cases, the compiler proof needs only the four properties below: digest correctness, owner-hidden exposure, same-element compatibility, and adaptive exactness.

**Owner-hidden digest interface  $\Pi_{\text{HDig}}$** 

An owner-hidden digest realization provides algorithms or interactive protocols:

1. **DigKeyReg** fixes a stable digest domain and a digest-key handle  $\text{ck}_{\text{dig}}$ .
2. **DigSetup** binds a committed set  $S$  to a private digest state  $\text{dst}_0$  and a public digest handle  $\text{dpub}_0$ .
3. **DigUpdate** transforms  $(\text{dst}_{t-1}, \text{dpub}_{t-1})$  into  $(\text{dst}_t, \text{dpub}_t)$  for a canonical update  $(\Delta_t^+, \Delta_t^-)$  using the same stable tags.

4. **DigSession** takes active records that have already passed syntax, no-injection, distinctness, and same-element-binding checks, and accepts only when their digest equals the digest represented by the current public handle.

The public handle is realization-dependent. For  $\Pi_{\text{HDig}}^{2\text{PC}}$  it consists of commitments to additive digest shares. For  $\Pi_{\text{HDig}}^{\text{ShareTag}}$  it consists of an owner-aggregate commitment  $D_t^P$ , a homomorphic encryption  $E_t^V$  of the verifier aggregate, and per-record authenticated encrypted-share handles used inside update/session relations. The compiler never uses the internal form of this handle except through **DigSession** and the security property below.

#### Required properties of an owner-hidden digest realization

Let

$$\gamma_x = \Gamma_{K_{\text{dig}}}(\text{digest} \parallel \text{applD} \parallel \text{partyID} \parallel \text{stateID} \parallel x)$$

be the stable digest tag for element  $x$ . A realization of  $\Pi_{\text{HDig}}$  must satisfy:

1. **Digest correctness.** For every honestly generated accepted state  $S_t$ , the public digest handle  $\text{dpub}_t$  and the private digest state  $\text{dst}_t$  represent the algebraic digest

$$A(S_t) = \sum_{x \in S_t} \gamma_x \in \mathbb{F}^\ell,$$

and accepted updates preserve

$$A(S_t) = A(S_{t-1}) + \sum_{x \in \Delta_t^+} \gamma_x - \sum_{x \in \Delta_t^-} \gamma_x.$$

2. **Owner-hidden exposure.** The state owner does not learn raw tags  $\gamma_x$ , tag-opening randomness, or linear predicates of hidden tags beyond the equality outcomes and abort stages explicitly included in the leakage of the digest realization.
3. **Same-element compatibility.** Every digest contribution used in setup, update, or session is bound to the same hidden element commitment  $C_x$  that is used by the compatible committed-token/no-injection relation. This property is what lets the compiler combine no-injection and digest equality.
4. **Adaptive exactness.** Let  $S$  be the set represented by an accepted digest state and let  $S_{\text{use}} \subseteq S$  be fixed before an equality attempt reaches **DigSession**. If **DigSession** accepts with  $S_{\text{use}} \subsetneq S$ , the event is bounded by a realization-specific advantage  $\text{Adv}_{\Pi}^{\text{hdig}}(\lambda)$ .



### Digest security experiment and advantage

The experiment  $\text{Exp}_{\Pi, A}^{\text{hdig}}(1^\lambda)$  gives the adversary the public digest handles and all leakage specified by the selected realization. The adversary adaptively submits session attempts. A counter  $q_{\text{dig}}$  counts only attempts that pass syntax, no-injection, distinctness, and same-element binding before reaching the digest equality test. The adversary wins if such an attempt is accepted for a strict subset  $S_{\text{use}} \subsetneq S$  of the state represented by the accepted digest handle. We define

$$\text{Adv}_{\Pi, A}^{\text{hdig}}(\lambda) = \Pr[\text{Exp}_{\Pi, A}^{\text{hdig}}(1^\lambda) = 1].$$

The generic and cached realizations below instantiate this advantage with different leakage and implementation costs. The Exact-PICS and HDUA theorems depend only on  $\text{Adv}_{\Pi}^{\text{hdig}}$ , not on the internal realization.

## 6 Generic 2PC Realization of Owner-Hidden Digest

The generic realization is the canonical baseline. It keeps no reusable per-element tag handles: whenever a digest is initialized, updated, or checked, the parties evaluate the stable-domain PRF inside malicious-secure arithmetic 2PC. This gives the cleanest leakage story, because the owner learns neither raw tags nor any digest predicate beyond the declared accept/reject output.

*Key registration and domain.* Before the committed state is fixed, the parties commit to key shares  $k_{\text{dig}}^P, k_{\text{dig}}^V$  and derive

$$K_{\text{dig}} = H_{\text{key}}(k_{\text{dig}}^P, k_{\text{dig}}^V, \text{appID}, \text{partyID}, \text{stateID}).$$

The digest tag for  $x$  is always

$$\gamma_x = F_{K_{\text{dig}}}(\text{digest} \parallel \text{appID} \parallel \text{partyID} \parallel \text{stateID} \parallel x),$$

with no session identifier or epoch handle in the digest domain.

*State and checks.* The digest value  $A_t = \sum_{x \in S_t} \gamma_x$  is additively shared as  $A_t^P + A_t^V$  and bound by public commitments  $D_t^P, D_t^V$ . **DigSetup** computes the initial shares in 2PC. **DigUpdate** checks openings of the previous share commitments, computes the insertion/deletion delta in 2PC, and refreshes the shares internally. **DigSession** verifies that the provided shares open the current commitments and checks

$$\sum_{x \in S_{\text{use}}} \gamma_x = A_t^P + A_t^V$$

inside the 2PC circuit. The complete circuits and the adaptive rejection analysis are given in Appendix B.2.

**Lemma 1 (Generic 2PC owner-hidden digest security).** *Assume the key-derivation experiment is secure,  $\Gamma$  is a PRF into  $\mathbb{F}^\ell$ , the share commitments are binding, and the arithmetic 2PC realizes the specified digest circuits with malicious security and the stated abort leakage. Then*

$$\text{Adv}_{\Pi_{2\text{PC}}}^{\text{hdig}} \leq \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_{\text{HDig}}^{2\text{pc}} + \text{Adv}_{\Gamma}^{\text{prf}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \frac{q_{\text{dig}}}{|\mathbb{F}|^\ell - q_{\text{dig}}}.$$

The formal game sequence, including the conditioning argument for rejected digest tests, is in Appendix B.2.

## 7 Cached Authenticated ShareTag Realization

The cached realization moves expensive PRF evaluation from the session path to setup and insertion time. The main hidden-state theorem uses encrypted verifier shares rather than public homomorphic tag commitments, because public tag handles allow offline subset-aggregate tests.

*TagShareGen.* For an element commitment  $C_x = \text{Com}_X(x; r_x^C)$ , the owner and verifier run a malicious-secure 2PC protocol **TagShareGen**. The owner inputs  $(x, r_x^C, C_x, k_{\text{dig}}^P)$  and the verifier inputs  $(k_{\text{dig}}^V, sk_{\text{tag}}, pk_V)$ . The circuit checks  $C_x = \text{Com}_X(x; r_x^C)$ , derives  $K_{\text{dig}}$ , and computes the stable tag

$$\gamma_x = \Gamma_{K_{\text{dig}}}(\text{digest}, \text{applD}, \text{partyID}, \text{stateID}, x).$$

It samples  $s_x^P \leftarrow \mathbb{F}^\ell$ , sets  $s_x^V = \gamma_x - s_x^P$ , and outputs to the owner only

$$\text{stag}_x^P = (\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, s_x^P, E_x^V, \mu_x^P), \quad E_x^V = \text{Enc}_{pk_V}(s_x^V).$$

The owner never receives  $s_x^V$  or  $\gamma_x$ . The authentication tag  $\mu_x^P$  binds the share tuple to  $C_x$ , the digest context, and the generated encrypted verifier share.

**Lemma 2 (Authenticated tag-share generation).** *Under malicious security of TagShareGen, key-derivation security, PRF security, binding of  $\text{Com}_X$ , semantic security and correctness of the encryption layer, and unforgeability of the share-authentication mechanism, every accepting share handle is bound to a unique committed element  $x$  and satisfies*

$$s_x^P + \text{Dec}_{sk_V}(E_x^V) = \Gamma_{K_{\text{dig}}}(x)$$

for the stable digest domain, except with

$$\text{Adv}^{\text{tagshare-gen}} + \text{Adv}^{\text{authshare}} + \text{Adv}^{\text{pke}} + \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_{\Gamma}^{\text{prf}} + \text{Adv}_{\text{Com}_X}^{\text{bind}}.$$

The owner share is statistically masked, and the verifier's 2PC view is simulatable without learning  $x$  beyond the allowed leakage.

The formal proof is in Appendix B.3.

*Session aggregate binding.* For active records, the owner aggregates owner shares and encrypted verifier shares. A relation  $R_{\text{ShareAggBind}}$  first verifies every authenticated share handle, checks same-element binding to the PSI token and accumulator record, and certifies

$$A_{\text{use}}^P = \sum_{x \in S_{\text{use}}} s_x^P, \quad E_{\text{use}}^V = \bigoplus_{x \in S_{\text{use}}} E_x^V,$$

with analogous checks for the epoch aggregates  $A_t^P, E_t^V$ . The values passed to  $\Pi_{\text{ShareAgg}}$  are bound by commitments, so the owner cannot prove one aggregate and input another.

*Aggregate equality.* The protocol  $\Pi_{\text{ShareAgg}}$  takes the bound aggregate commitments and ciphertexts as public inputs. The owner opens the owner-aggregate commitments, the verifier inputs  $sk_V$ , and the 2PC circuit decrypts only the aggregate ciphertexts and checks

$$A_{\text{use}}^P + \text{Dec}_{sk_V}(E_{\text{use}}^V) = A_t^P + \text{Dec}_{sk_V}(E_t^V).$$

The verifier does not learn active record identities, and the owner cannot run uncounted full-digest equality tests because it lacks the verifier shares and cannot decrypt the ciphertexts.

**Lemma 3 (Cached authenticated shares realize owner-hidden digest).**

*Assume Lemma 2, soundness of same-element binding and  $R_{\text{ShareAggBind}}$ , malicious security of  $\Pi_{\text{ShareAgg}}$ , and the encryption and commitment assumptions above. Then the cached ShareTag realization satisfies the owner-hidden digest interface with*

$$\begin{aligned} \text{Adv}_{\Pi_{\text{ShareTag}}}^{\text{hdig}} &\leq \text{Adv}^{\text{tagshare-gen}} + \text{Adv}^{\text{shareagg}} + \text{Adv}^{\text{aggbind}} + \text{Adv}^{\text{authshare}} \\ &\quad + \text{Adv}^{\text{pke}} + \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_\Gamma^{\text{prf}} + \text{Adv}_{\text{Com}}^{\text{bind}} \\ &\quad + \frac{q_{\text{dig}}}{|\mathbb{F}|^\ell - q_{\text{dig}}}. \end{aligned}$$

The formal proof is in Appendix B.4.

*Public-handle variant.* Public homomorphic tag commitments are only a leakage-explicit deployment variant. They are not used in the hidden-state theorem: any public aggregate-test visibility must be included in leakage or counted in a separate  $q_{\text{alltest}}$  budget.

## 8 Exact-PICS Compiler

The Exact-PICS compiler leaves the underlying PSI computation essentially unchanged. It adds one persistent digest state during commitment and one digest equality check before each PSI session. The base protocol still supplies no-injection and token privacy. The compiler’s new security role is to turn the subset guarantee  $S_{\text{use}} \subseteq S$  into equality.

## 8.1 Syntax

The compiler takes a compatible NI-ctok-PICS protocol and adds  $\Pi_{\text{HDig}}$ .

**Construction:**  $\text{ExactPICS}[\text{NI-ctok-PICS}, \Pi_{\text{HDig}}]$

1. **Commit.** The committer runs the base committed-set setup, obtaining a commitment handle  $C_S$  and a setup proof. In parallel, the parties run **DigSetup** on the same committed elements to obtain a digest public handle  $\text{dpub}_S$  and any realization-specific private digest state. The reusable set handle is

$$\eta_S = (C_S, \text{dpub}_S, n, h_S),$$

where  $h_S = H(C_S, \text{dpub}_S, n, \text{ck}_{\text{dig}}, \text{pp})$ . For  $\Pi_{\text{HDig}}^{2\text{PC}}$ ,  $\text{dpub}_S = (D_S^P, D_S^V)$ ; for  $\Pi_{\text{HDig}}^{\text{ShareTag}}$ ,  $\text{dpub}_S = (D_S^P, E_S^V)$  together with cached authenticated encrypted-share handles used inside the session relation.

2. **Session.** For a PSI session, the committer submits active committed-token records and the base no-injection proof. The parties run **DigSession** for the chosen digest realization on the active records and stored digest state. The PSI backend is invoked only if the no-injection proof, distinctness/same-element checks, and digest check all accept.

The compiled session accepts only when two independent checks agree. The base PICS-like relation says that the active records are a subset of the committed set. The digest relation says that the active records have the same hidden linear fingerprint as the committed set. Together these imply exact use.

## 8.2 Security experiment

**Experiment**  $\text{Exp}_{\text{ExactPICS}, A}^{\text{exact}}(1^\lambda)$

1. The challenger generates public parameters and gives the public handles to  $A$ .
2.  $A$  produces a malicious commit transcript. If setup rejects, the experiment returns 0. Otherwise the setup extractor fixes a canonical committed set  $S$ .
3.  $A$  adaptively starts session attempts. For every accepting session, the session extractor obtains  $S_{\text{use}}$ .
4.  $A$  wins if any accepted session satisfies  $S_{\text{use}} \neq S$ .

The leakage includes  $n$ , session identifiers, public abort stages, and the authorized PSI output, but not nonintersection elements or raw digest tags.

This theorem is the paper's main result. All later dynamic constructions are applications that provide a suitable no-injection layer for evolving states.

**Theorem 1 (Exact-PICS compiler).** *Let the base protocol be a compatible NI-ctok-PICS satisfying setup extraction, session no-injection, active-record distinctness, same-element binding, and token privacy. Let  $\Pi_{\text{HDig}}$  satisfy owner-hidden digest security with advantage  $\text{Adv}_{\Pi}^{\text{hdig}}$ . Then the compiled protocol realizes exact reusable committed-set PSI against malicious committers with*

$$\begin{aligned} \text{Adv}_{\text{ExactPICS},A}^{\text{exact}}(\lambda) &\leq \text{Adv}_{\text{NI-ctok-PICS}}^{\text{setup}}(\lambda) + \text{Adv}_{\text{NI-ctok-PICS}}^{\text{noinj}}(\lambda) \\ &\quad + \text{Adv}_{\text{NI-ctok-PICS}}^{\text{sebind}}(\lambda) + \text{Adv}_{\Pi}^{\text{hdig}}(\lambda). \end{aligned}$$

Instantiating  $\Pi$  with  $\Pi_{\text{HDig}}^{2\text{PC}}$  or  $\Pi_{\text{HDig}}^{\text{ShareTag}}$  gives the concrete bounds of Lemmas 1 and 3.

*Proof.* The full bad-event proof is given in Appendix B.5. It decomposes a winning transcript into setup-extraction failure, no-injection failure, same-element-binding failure, or an accepting strict-subset digest query, and bounds the final case by the owner-hidden digest theorem.

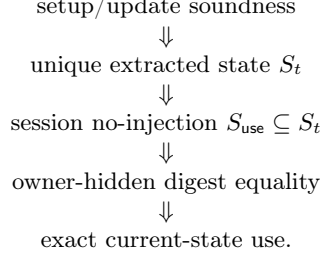
## 9 Realizability Backend and Dynamic Application

The compiler is not black-box for arbitrary PSI. This section records a simple backend showing that the compatible interface is realizable, and then describes the dynamic hidden-state application used later. The backend is a witness for the abstraction, not a performance-optimized PICS system.

The compatibility map in Section 4 explains why this backend satisfies the interface and why arbitrary PSI transcripts do not. The point of the backend is realizability: it shows that the compiler is not vacuous, while optimized integrations with concrete PICS-style protocols can replace it whenever they expose the same setup-extraction, active-record, no-injection, and same-element-binding hooks.

*Static backend.* The static backend commits to the setup set using a bilinear set-polynomial accumulator. A session record is accepted only if it carries a hidden membership witness under this accumulator. Same-element binding links the membership representative, element commitment, PSI token, and digest/share handle. Therefore the backend supplies  $S_{\text{use}} \subseteq S$  without enumerating all setup commitments in every session. The formal realizability proof is in Appendix B.6.

*Dynamic hidden-state application.* HDUA-LAUSE is the dynamic version of the same idea. The state owner maintains a hidden evolving set  $S_t$ . Accepted setup and updates extract to a unique state sequence; session membership proofs give  $S_{\text{use}} \subseteq S_t$ ; the owner-hidden digest then upgrades no-injection to  $S_{\text{use}} = S_t$ .



## 10 Bilinear Dynamic Instantiation

The bilinear instantiation represents a set by the roots of a polynomial. Each element  $x$  is mapped to  $a_x = H_{\text{elt}}(x)$  and

$$P_S(X) = \prod_{x \in S} (X - a_x), \quad A_S = g_1^{P_S(\tau)}.$$

The raw accumulator is deterministic and is therefore hidden behind a binding/hiding commitment  $C_S^{\text{acc}}$ .

*Membership and nonmembership.* For  $x \in S$ , the witness is  $W_x = g_1^{P_S(\tau)/(\tau - a_x)}$  and the pairing equation is

$$e(A_S, g_2) = e(W_x, g_2^\tau / g_2^{a_x}).$$

For  $y \notin S$ , the proof uses  $v_y = P_S(a_y) \neq 0$  and a quotient witness satisfying

$$e(A_S / g_1^{v_y}, g_2) = e(W_y, g_2^\tau / g_2^{a_y}), \quad v_y \cdot v_y^{-1} = 1.$$

*Batch updates.* For insertions  $I$  and deletions  $D$ , canonical update semantics are expressed by

$$P_{S_t}(X)P_D(X) = P_{S_{t-1}}(X)P_I(X),$$

which is checked at the hidden SRS point by

$$e(A_{t-1}, g_2^{P_I(\tau)}) = e(A_t, g_2^{P_D(\tau)}).$$

The relation  $R_{\text{BatchPoly}}$  proves that  $P_I, P_D$  and the group elements  $g_2^{P_I(\tau)}, g_2^{P_D(\tau)}$  are computed from the same hidden representatives used in membership, non-membership, update, and digest-binding checks.

*Concrete proof-system interface.* The selected instantiation is a hybrid commit-and-prove interface. Arithmetic statements are handled by a knowledge-sound arithmetic layer, while pairing-product equations are handled by a Groth-Sahai-style pairing layer with equation soundness. The two layers are tied by binding scalar-link commitments and linked group handles.

| Relation/object              | Statement checked                                                                                                          | Security role                                           |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Scalar commitment            | $L_a = g_1^a h_1^r$ for representatives, coefficients, arithmetic extractor obtains $a, r$ shares                          |                                                         |
| Group handle                 | $U_a = g_2^a$                                                                                                              | used natively in pairing equations                      |
| Link proof $R_{\text{link}}$ | $L_a$ and $U_a$ encode the same scalar $a$                                                                                 | prevents cross-layer substitution                       |
| Membership                   | $e(A_S, g_2) = e(W_x, g_2^\tau / U_{a_x})$                                                                                 | proves $a_x$ is a root of $P_S$                         |
| Nonmembership                | $e(A_S / g_1^v, g_2) = e(W, g_2^\tau / U_a)$ and $v \cdot v^{-1} = 1$                                                      | proves nonzero remainder                                |
| Batch polynomial             | arithmetic layer extracts coefficients of $P_I, P_D$ and checks $G_I = \prod_j (g_2^{\tau_j})^{c_j}$ , similarly for $G_D$ | binds update equations to hidden update representatives |
| Digest-input binding         | binds the token commitment, accumulator representative, ShareTag handle, and digest input all name the same $C_x$          | re-supplies same-element compatibility                  |

The composability boundary is explicit: extraction is claimed only for the arithmetic commit-and-prove layer and the scalar openings it controls; the pairing layer contributes soundness of the linked bilinear equations. All theorem bounds therefore include both  $\text{Adv}_{\text{arith}}^{\text{ks}}$  and  $\text{Adv}_{\text{pair}}^{\text{snd}}$ , plus the link-binding term.

*Binding assumption.* The bounded polynomial evaluation binding property is not intended as a new custom assumption. It is a named intermediate form of standard bounded-degree KZG binding.

**Lemma 4 (BPEB from KZG binding).** *Let the SRS contain  $(g_1, g_1^\tau, \dots, g_1^{\tau^q})$  for hidden  $\tau$ , and let  $N_{\max} \leq q$ . In the algebraic group model, any PPT adversary that outputs a nonzero polynomial  $Q$  of degree at most  $N_{\max}$  with  $g_1^{Q(\tau)} = 1$  yields an adversary against the usual bounded-degree KZG polynomial-binding, equivalently the corresponding  $q$ -SDH-style binding assumption, up to the standard algebraic reduction loss.*

The formal reduction is in Appendix B.1; the HDUA uniqueness proof invokes this lemma only after the arithmetic extractor has produced the two competing set polynomials.

## 11 Security Results

The security results compose in layers. The digest theorem proves no-omission once no-injection is available. The Exact-PICS theorem applies this to static committed-token PSI. The bilinear HDUA theorem proves that the dynamic accumulator supplies valid evolving states and session no-injection. The final session-exactness and VA-UPSI theorems then apply the digest theorem to the dynamic setting.

| Result              | Uses                                 | Provides                         |
|---------------------|--------------------------------------|----------------------------------|
| Owner-hidden digest | 2PC or cached tag-share equalization | no-omission from digest equality |
| Exact-PICS          | NI-ctok-PICS + digest                | exact reusable-set use           |
| Bilinear HDUA       | sound-accumulator + proofs           | valid evolving hidden state      |
| session exactness   | no-injection + digest                | exact current-state use          |
| VA-UPSI composition | HDUA exactness + PSI                 | exact-state VA-UPSI              |

**Theorem 2 (Bilinear HDUA setup and update soundness).** *Assume binding of  $\text{Com}_{G_1}$ , collision resistance of  $H_{\text{elt}}$ , knowledge soundness of the selected proof system, and binding of the set-polynomial accumulator under the bounded polynomial evaluation binding assumption for the SRS. Then every accepted setup and statefully accepted update sequence extracts to a unique canonical state sequence  $S_0, S_1, \dots, S_t$  satisfying the declared update semantics, except with*

$$\text{Adv}_{\text{bilHDUA}}^{\text{upd}} \leq \text{Adv}^{\text{BPEB}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \text{Adv}_{\text{arith}}^{\text{ks}} + \text{Adv}_{\text{pair}}^{\text{snd}} + \text{Adv}_{\text{link}}^{\text{bind}} + \text{Adv}_{H_{\text{elt}}}^{\text{coll}}.$$

*Proof.* The full induction and reduction to BPEB appear in Appendix B.7. The proof extracts setup and update witnesses, proves uniqueness of every extracted state by reducing any two-state opening to a nonzero polynomial  $Q$  with  $g_1^{Q(\tau)} = 1$ , and then proves by induction that each accepted update implements exactly the declared insertion/deletion transition.

**Theorem 3 (Digest-native session exactness).** *Assume bilinear HDUA setup/update soundness, session no-injection soundness, same-element binding, owner-hidden digest security of  $\Pi_{\text{HDig}}$  with advantage  $\text{Adv}_{\Pi}^{\text{hdig}}$ , and active-record distinctness. Then every accepted session at epoch  $t$  uses exactly the extracted state  $S_t$ , except with*

$$\text{Adv}_{\text{HDUA-LAUSE}}^{\text{exact}} \leq \text{Adv}_{\text{bilHDUA}}^{\text{upd}} + \text{Adv}_{\text{sess}}^{\text{noinj}} + \text{Adv}^{\text{sebind}} + \text{Adv}_{\Pi}^{\text{hdig}}.$$

Using the generic 2PC or cached tag-share digest realization substitutes the corresponding concrete bound for  $\text{Adv}_{\Pi}^{\text{hdig}}$ .

*Proof.* The full reduction is in Appendix B.8. It conditions on update soundness, no-injection, same-element binding, and distinctness. Any remaining accepted non-exact session is an accepting strict-subset digest query and is bounded by the owner-hidden digest theorem.

**Theorem 4 (Hidden-state privacy).** *Assume zero knowledge of the setup/update/session proofs, hiding of accumulator commitments and digest public handles, malicious privacy or hiding of the selected  $\Pi_{\text{HDig}}$  realization, token privacy of the PSI backend, and leakage that includes update sizes, session active-record counts, abort stages, abort roles, and authorized output type. Then two hidden-state histories with identical leakage induce computationally indistinguishable public views.*

The formal privacy experiment and hybrid proof are in Appendix B.9. The simulator replaces proofs by simulated proofs, accumulator and digest handles by hiding dummy handles, digest transcripts by the corresponding 2PC or ShareTag simulators, and committed-token PSI transcripts by the PSI simulator. Thus the final view depends only on the declared leakage and authorized output.

The master bounds in the appendix are intentionally additive. Each term corresponds to a way an accepted transcript could avoid the intended invariant: bad key derivation, bad digest pseudorandomness, bad 2PC simulation, failed proof extraction, invalid pairing equation, broken link binding, accumulator collision, representative collision, or a strict omitted set passing the digest equality test.



## 12 VA-UPSI Corollary

VA-UPSI is an application of the exact-state layer rather than a separate source of exactness. Each party first authenticates its evolving hidden state using the HDUA-LAUSE application above. A flagged committed-token PSI backend then computes the authorized intersection functionality on records that are already bound to the exact current states.

**Corollary 1 (VA-UPSI application).** *Assume the digest-native HDUA exactness theorem for both parties’ state-authentication layers and a flagged committed-token shared-output PSI backend realizing its standard ideal functionality under static malicious corruption of at most one party. Then the composed VA-UPSI protocol realizes the exact-state VA-UPSI functionality with leakage consisting of state sizes, update sizes, active-record counts, abort stage and role, and the authorized output.*

The detailed ideal functionality and simulator are given in Appendix B.10. The simulator extracts accepted state histories and active session records, applies digest-native exactness to identify the active records with the current state, and then invokes the flagged-PSI simulator. Adaptive corruptions and global multi-verifier non-equivocation are outside this baseline corollary; global non-equivocation requires an external transparency log.

## 13 Efficiency and Comparison

The costs below should be read as authentication overheads, not as full PSI runtime. The PSI backend still determines the live-query cost. The tables use short cost codes; explanatory prose is kept outside the tabular bodies.

**Table 1.** Compact authentication overhead.

| Component          | Setup                 | Update                  | Session / query          |
|--------------------|-----------------------|-------------------------|--------------------------|
| Generic 2PC digest | $n_0 C_\Gamma$ in 2PC | $u_t C_\Gamma$ in 2PC   | $m C_\Gamma$ in 2PC + EQ |
| Cached ShareTag    | $n_0$ TG              | $ \Delta_t^+ $ TG + Agg | $m$ Agg + AB + ADE       |
| Bilinear NI layer  | set polynomial        | batch update proof      | $m$ Mem + SEB            |
| Enum. baseline     | representation        | update certs            | full-handle coverage     |
|                    | certs                 |                         |                          |

TG = tag-share generation; Agg = owner-share/ciphertext aggregation; AB = aggregate-binding proof; ADE = aggregate-decryption equality 2PC; EQ = digest-equality output.

The digest-native route is most helpful when exactness would otherwise require proving complete coverage of a larger certified representation, such as a historical base/delete/add view or a long list of certified handles. It does not

reduce the number of live records processed by the PSI backend. It changes the authentication mechanism: the protocol proves no-injection for active records and then uses owner-hidden digest equality to rule out omissions.

The modular digest interface makes the main tradeoff explicit. The generic  $\Pi_{\text{HDig}}^{2\text{PC}}$  realization is the simplest baseline: it evaluates PRF tags inside malicious-secure arithmetic 2PC whenever a digest is initialized, updated, or checked. The cached  $\Pi_{\text{HDig}}^{\text{ShareTag}}$  realization is the optimized path: each element receives an authenticated owner share and an encrypted verifier-share handle once when it enters the committed state, and later sessions check exactness by aggregating owner shares and ciphertexts and running one aggregate-decryption equality 2PC. Thus the two realizations prove the same owner-hidden digest property but place the PRF cost on different protocol paths.

**Table 2.** Digest realization choices.

| Realization | PRF placement          | Session check      | Status         |
|-------------|------------------------|--------------------|----------------|
| 2PC         | setup, update, session | $mC_F$ in 2PC      | baseline       |
| ShareTag    | setup and insertions   | Agg + AB + ADE     | theorem path   |
| PubCTag     | setup and insertions   | public aggregation | leakage option |

Rows abbreviate the three digest realizations defined above. The public-handle option is separate from the cached ShareTag theorem variant.

The table deliberately separates the theorem-level optimized realization from the public-handle deployment option. The cached ShareTag theorem gives the malicious owner only random owner shares and encrypted verifier shares, and the verifier decrypts only aggregate ciphertexts inside 2PC; this preserves the protocol-visible equality-test accounting. The PubCTag deployment gives the owner public composable handles and is useful only when its extra aggregate-test visibility is acceptable or explicitly counted.

### 13.1 Cost model

Let  $n_0 = |S_0|$ ,  $u_t = |\Delta_t^+| + |\Delta_t^-|$ , and  $m$  be the number of active records in a session. Let  $C_F$  be the arithmetic cost of one field-native PRF evaluation,  $C_H$  the cost of one hash-to-field computation, and  $C_{2\text{PC}}(g)$  the communication/online cost of malicious-secure 2PC for an arithmetic circuit with  $g$  multiplication gates. A Poseidon- or Rescue-style field-native PRF is the intended instantiation of  $F$ ; the exact constants depend on the chosen field, width, and number of rounds.

**Table 3.** Cost drivers for digest-native bilinear HDUA-LAUSE.

| Component            | Driver                                                              |
|----------------------|---------------------------------------------------------------------|
| Common proof work    | $u_t C_H + u_t \log u_t$ for updates; $m C_H$ plus SEB for sessions |
| 2PC digest           | $C_{2PC}(n_0 C_R)$ , $C_{2PC}(u_t C_R)$ , and $C_{2PC}(m C_R)$      |
| ShareTag digest      | $n_0$ TG, $ \Delta_t^+ $ TG, and $m$ Agg + AB + ADE                 |
| Batch polynomial     | naive $O(u_t^2)$ or product-tree $O(u_t \log^2 u_t)$                |
| Bilinear update      | one pairing-product equation plus Mem/NonMem proofs                 |
| Session no-injection | $m$ Mem + $m$ SEB over active records                               |
| SRS and state        | $O(N_{\max})$ SRS; constant public epoch state, excluding proofs    |
| PSI backend          | backend-dependent over active records; not sublinear live PSI       |

TG, Agg, AB, ADE, Mem, and SEB are as defined in Table 1.

**Table 4.** Concrete parameter choices for estimates.

| Item                | Target / estimate role                                                                 |
|---------------------|----------------------------------------------------------------------------------------|
| Field and groups    | BLS12-381 scalar field; asymmetric pairing; $ \mathbb{F}  \approx 2^{255}$             |
| Digest dimension    | $\ell \in \{1, 2\}$ ; error term $q_{\text{dig}}/( \mathbb{F} ^\ell - q_{\text{dig}})$ |
| PRF                 | field-native PRF; cost unit $C_R$                                                      |
| 2PC digest gates    | $n_0 C_R$ , $u_t C_R$ , and $m C_R$                                                    |
| ShareTag digest     | TG + Agg + AB + ADE                                                                    |
| SRS and pairings    | $O(N_{\max})$ SRS powers; batch/update equations                                       |
| Comparison baseline | complete historical-handle enumeration                                                 |

The cached ShareTag variant is the optimized realization used in the main hidden-state theorem. The public-handle PubCTag row is a separate leakage-explicit deployment option, not an instantiation of the ShareTag theorem. Its leakage is different and it is not silently substituted into the main theorem.

The construction avoids complete historical base/delete/add enumeration as the exactness proof. It still processes the live records used by the PSI backend. Compared with complete-enumeration LAUSE, the security bottleneck moves from enumerating every certified handle in the representation to proving per-record no-injection and a single hidden digest equality.

The following table is analytic rather than experimental. Its purpose is to show how the overhead scales with setup size  $n_0$ , update size  $u_t$ , and active session size  $m$ , and to make visible where the cached ShareTag realization saves session work.

**Table 5.** Illustrative analytic workloads.

| Scenario | $u_t$ | $m$    | $\ell$ | 2PC update | 2PC session           | ShareTag session      |
|----------|-------|--------|--------|------------|-----------------------|-----------------------|
| Small    | 10    | $10^3$ | 1      | 10 PRFs    | $10^3$ PRFs           | $10^3$ Agg + AB + ADE |
| Medium   | 100   | $10^4$ | 1      | 100 PRFs   | $10^4$ PRFs           | $10^4$ Agg + AB + ADE |
| Large    | 1000  | $10^5$ | 2      | 1000 PRFs  | $2 \cdot 10^5$ coords | $10^5$ Agg + AB + ADE |

Counts are protocol-level cost drivers, not wall-clock benchmarks; ShareTag tag generation is paid on insertions.

### 13.2 Prototype digest measurements

To complement the analytic model, we measured the digest-session path on eight matched parameter points with  $n_0 \in \{100, 1000\}$ ,  $u_t \in \{10, 100\}$ ,  $m \in \{100, 1000\}$ , and  $\ell = 1$ . The measurements isolate the digest layer; they are not full PSI or full HDUA-LAUSE benchmarks. Table 6 therefore separates the modeled lower bound, a live MP-SPDZ digest-equality probe, and the cached ShareTag session path.

**Table 6.** Prototype digest-session measurements.

| Path              | Range          | Median   | Interpretation             |
|-------------------|----------------|----------|----------------------------|
| Modeled ideal 2PC | 0.10–1.53 ms   | 0.19 ms  | arithmetic lower bound     |
| Live MP-SPDZ 2PC  | 156.5–250.2 ms | 218.1 ms | live digest-equality probe |
| Cached ShareTag   | 1.58–12.33 ms  | 3.71 ms  | cached online digest path  |

The full matched rows appear in Appendix A; the first row is not a live 2PC runtime.

Across the eight matched points, cached ShareTag reduces online digest-session latency by  $17.8\times$ – $137.5\times$  relative to the live MP-SPDZ digest-equality probe, with median reduction  $50.9\times$ . This is a prototype-level statement about where the session cost goes: the live probe includes fixed runtime and orchestration overhead, each point has a single timing, and only  $\ell = 1$  was measured. The conclusion is not that cached ShareTag is faster than every optimized 2PC backend, but that moving digest tag generation and equality work off the common session path is concretely useful in the tested implementation.

### 13.3 Proof-size and operation accounting

In the chosen hybrid proof model, the pairing-product layer contributes a constant number of Groth-Sahai-style proof elements per pairing equation, while the arithmetic layer contributes a proof for coefficient construction, distinctness, hash-to-field, and same-element binding. The dominant non-digest terms are

$$\begin{aligned} \text{update arithmetic size} &\approx O(u_t C_H + u_t \log u_t), \\ \text{session arithmetic size} &\approx O(m C_H) + \text{same-element/link checks}. \end{aligned}$$

For  $\Pi_{\text{HDig}}^{2\text{PC}}$ , add  $O(mC_I)$  session digest-circuit work. For the cached encrypted-share variant  $\Pi_{\text{HDig}}^{\text{ShareTag}}$ , replace that term by aggregation of  $m$  authenticated owner shares and verifier-share ciphertexts, the relation  $R_{\text{ShareAggBind}}$ , one aggregate-decryption equality 2PC, and authentication/link verification inside the session proof. This removes session-time PRF evaluation from the digest path, but it does not make digest checking free.

If public homomorphic tag handles are used in the separate PubCTag deployment option, aggregation is cheaper, but aggregate-test visibility must be included in the leakage or counted in the digest-query budget  $q_{\text{alltest}}$ . If a pure SNARK is used instead of the hybrid path, pairing checks must either be verified non-natively or moved outside the circuit, which increases constants substantially. The hybrid proof path is selected precisely to avoid a large non-native-pairing circuit.

### 13.4 Feature comparison

**Table 7.** Security and functionality positioning.

| Line           | Dyn.   | Hidden | Exact use | Deletions | Cross-session |
|----------------|--------|--------|-----------|-----------|---------------|
| Malicious PSI  | No     | No     | One run   | N/A       | No            |
| PICS           | Static | Yes    | Interface | N/A       | Yes           |
| UPSI / revised | Yes    | Model  | No        | Yes       | Model         |
| Enum. LAUSE    | Yes    | Yes    | Yes       | Yes       | Stateful      |
| Digest-native  | Yes    | Yes    | Yes       | Yes       | Stateful      |

**Table 8.** Efficiency and deployment positioning.

| Line           | Authentication cost         | Query focus               | Assumption             |
|----------------|-----------------------------|---------------------------|------------------------|
| PICS           | static consistency proofs   | committed/reusable inputs | compatible interface   |
| UPSI / revised | backend-specific            | update-dependent PSI      | backend model          |
| Enum. LAUSE    | full handle enumeration     | base/delete/add view      | stateful verifier      |
| Digest-native  | NI + hidden digest equality | active live records       | SRS; stateful verifier |

Global non-equivocation for Digest-native still requires an external append-only log or transparency service.

The most important comparison is with complete enumeration. Complete enumeration proves exactness by forcing every certified historical handle to appear.

The digest-native route proves only no-injection for active records and uses the hidden digest to rule out omissions. This changes the authentication mechanism, not the fact that the PSI engine must process its active inputs.

## 14 Limitations

*Not a faster PSI backend.* The construction is an exact-state authentication layer, not a faster UPSI engine. The PSI backend still processes the active live records in a query. The digest-native construction replaces complete enumeration of the historical certified representation by no-injection plus hidden digest equality.

*Structured reference string and freshness.* The bilinear instantiation uses a structured reference string up to degree  $N_{\max}$ . The main rollback/fork guarantee is local to a stateful verifier that stores the latest accepted epoch handle for the counterparty. Public non-equivocation across many verifiers requires an external append-only log or transparency service.

*Digest-realization caveats.* Cached ShareTag is an optimization of the owner-hidden digest layer, not a new exactness principle. It preserves the equality-leakage model used in the main theorem because the malicious owner receives only random owner shares and encrypted verifier-share handles; aggregate decryption occurs only inside  $\Pi_{\text{ShareAgg}}$ , so the owner cannot test full subset aggregates offline. Public homomorphic tag commitments form a separate cheaper deployment variant, but they expose aggregate-test visibility that must be included in leakage or counted in a digest-query budget  $q_{\text{alltest}}$ ; they are not used in the hidden-state theorem.

*Assumption diversity.* RSA accumulators may provide an alternative HDUA backend, but hidden hash-to-prime and hidden nonmembership proofs introduce additional machinery; we leave this direction outside the present work.

*Hybrid proof model.* The proof system is deliberately hybrid. The arithmetic layer is the knowledge-extractable layer. The Groth-Sahai-style layer supplies soundness for pairing-product equations linked to extracted scalar commitments. This avoids non-native pairing verification inside a single large circuit, but it requires concrete link commitments and link soundness as stated in the assumptions.

## 15 Conclusion

We gave a compiler from compatible no-injection committed-token PSI to exact reusable committed-set PSI. The compiler uses same-element binding and an owner-hidden digest interface to turn subset consistency into exact consistency. We gave both a generic 2PC realization and a cached authenticated tag-share realization of the digest layer. The dynamic HDUA-LAUSE application uses a hidden bilinear accumulator for update validity and no-injection, and the same

digest principle for no-omission. Under the stated proof-system, accumulator-binding, 2PC, PRF, and commitment assumptions, every accepted session uses exactly the extracted committed or current state, except with the explicit adaptive digest term  $q_{\text{dig}}/(|\mathbb{F}|^\ell - q_{\text{dig}})$  and component-security losses. The broader message is that subset consistency is a useful intermediate property: once no-injection is available, a carefully hidden and same-element-bound digest can upgrade it to exactness.

## References

1. Agarwal, A., Cash, D., George, M., Kamara, S., Moataz, T., Singh, J.: Updatable private set intersection from structured encryption. *Cryptology ePrint Archive*, Paper 2024/1183 (2024)
2. Badrinarayanan, S., Miao, P., Xie, T.: Updatable private set intersection. *Proceedings on Privacy Enhancing Technologies* 2022(2), 378–406 (2022)
3. Badrinarayanan, S., Miao, P., Shi, X., Tromanhauser, M., Zeng, R.: Updatable private set intersection revisited: Extended functionalities, deletion, and worst-case complexity. *ASIACRYPT* 2024, LNCS 15489, 200–233 (2024)
4. Camenisch, J., Lysyanskaya, A.: Dynamic accumulators and application to efficient revocation of anonymous credentials. *CRYPTO* 2002, LNCS 2442, 61–76 (2002)
5. Campanelli, M., Fiore, D., Han, S., Kim, J., Kolonelos, D., Oh, H.: Succinct zero-knowledge batch proofs for set accumulators. *CCS* 2022, 2605–2620 (2022)
6. Catalano, D., Fiore, D.: Vector commitments and their applications. *PKC* 2013, LNCS 7778, 55–72 (2013)
7. Merkle, R.C.: A digital signature based on a conventional encryption function. *CRYPTO* 1987, LNCS 293, 369–378 (1988)
8. Benarroch, D., Fuchsbauer, G., Malkin, T., Miers, I., Tromer, E.: Zero-knowledge proofs for set membership: efficient, succinct, modular. *EUROCRYPT* 2021, LNCS 12696, 393–422 (2021)
9. Garimella, G., Pinkas, B., Rosulek, M., Trieu, N., Yanai, A.: Oblivious key-value stores and amplification for private set intersection. *CRYPTO* 2021, LNCS 12826, 395–425 (2021)
10. Goel, A., Miao, P., Pham, P.V.L., Singh, S.: PICS: Private intersection over committed (and reusable) sets. *Cryptology ePrint Archive*, Paper 2025/1071 (2025)
11. Groth, J., Sahai, A.: Efficient non-interactive proof systems for bilinear groups. *EUROCRYPT* 2008, LNCS 4965, 415–432 (2008)
12. Damgård, I., Pastro, V., Smart, N.P., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. *CRYPTO* 2012, LNCS 7417, 643–662 (2012)
13. Keller, M.: MP-SPDZ: A versatile framework for multi-party computation. *CCS* 2020, 1575–1590 (2020)
14. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. *ASIACRYPT* 2010, LNCS 6477, 177–194 (2010)
15. Li, J., Li, N., Xue, R.: Universal accumulators with efficient nonmembership proofs. *ACNS* 2007, LNCS 4521, 253–269 (2007)
16. Pinkas, B., Rosulek, M., Trieu, N., Yanai, A.: PSI from PaXoS: Fast, malicious private set intersection. *EUROCRYPT* 2020, LNCS 12106, 739–767 (2020)
17. Rindal, P., Schoppmann, P.: VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. *EUROCRYPT* 2021, LNCS 12697, 901–930 (2021)
18. Srinivasan, S., Karantaidou, I., Papamanthou, C.: Batching, aggregation, and zero-knowledge proofs in bilinear accumulators. *CCS* 2022, 2597–2604 (2022)

## A Prototype Measurement Details

This appendix records the eight matched digest-session measurements summarized in Table 6. The ideal row is a modeled arithmetic lower bound, not a live runtime. The MP-SPDZ row is the live digest-equality probe. The ShareTag row is the cached online digest path. All live probe sessions accepted.

**Table 9.** Matched digest-session measurements.

| $n_0$ | $u_t$ | $m$  | $\ell$ | Ideal 2PC ms | MP-SPDZ ms | ShareTag ms | MP-SPDZ/ShareTag |
|-------|-------|------|--------|--------------|------------|-------------|------------------|
| 100   | 10    | 100  | 1      | 0.10         | 217.23     | 1.58        | 137.5            |
| 100   | 10    | 1000 | 1      | 0.16         | 208.79     | 3.43        | 60.8             |
| 100   | 100   | 100  | 1      | 0.15         | 220.90     | 3.60        | 61.3             |
| 100   | 100   | 1000 | 1      | 0.26         | 156.50     | 3.81        | 41.0             |
| 1000  | 10    | 100  | 1      | 0.22         | 201.70     | 5.76        | 35.0             |
| 1000  | 10    | 1000 | 1      | 1.53         | 218.87     | 12.33       | 17.8             |
| 1000  | 100   | 100  | 1      | 0.13         | 221.31     | 2.53        | 87.6             |
| 1000  | 100   | 1000 | 1      | 0.95         | 250.16     | 7.30        | 34.3             |

The last column is the ratio between the live MP-SPDZ probe and the cached ShareTag path.

The broader prototype CSV also contains full HDUA/session and end-to-end timings. We keep those numbers out of the main efficiency claim because they include unoptimized accumulator, membership, and no-injection machinery rather than isolating digest exactness. They are nevertheless useful diagnostics: in the attached results, `hdua_session` ranges from 3.39 s to 528.05 s, while `end_to_end_scenario` ranges from 4.80 s to 660.90 s. This supports the interpretation that the dominant remaining prototype cost is the no-injection/membership layer, not the hidden digest check.

For reproducibility, the CSV records Linux 6.8 on an Intel Core i7-1355U host with `rustc 1.91.1`; the recorded git commit hash is `unknown`. A final artifact should replace that placeholder by a commit identifier.

## B Formal Security Proofs

This appendix gives the full game-based proofs used in the main text. All probabilities are over the coins of the challenger, adversary, setup algorithms, proof systems, 2PC protocols, and random oracles/KDFs when the corresponding assumption is stated in that model. Every experiment returns 1 exactly when the adversary wins the named security game.

The appendix follows the dependency structure of the main text. Appendix B.2 proves the generic 2PC hidden digest theorem and Appendix B.4 proves the cached authenticated tag-share realization. Appendix B.5 uses the abstract digest advantage to prove the Exact-PICS compiler. Appendix B.6 shows that the static bilinear backend realizes the compatible no-injection interface. Appendices B.7 and B.8 prove dynamic update and session exactness. Appendices B.9 and B.10



give privacy and VA-UPSI composition. Each proof states its bad events and reduces them to one of the assumptions in Appendix B.1.

### B.1 Assumptions and advantage functions

For a commitment scheme  $\text{Com}$ ,  $\text{Adv}_{\text{Com}}^{\text{bind}}(B)$  and  $\text{Adv}_{\text{Com}}^{\text{hide}}(B)$  denote the standard binding and hiding advantages. For the digest PRF  $\Gamma$ ,  $\text{Adv}_{\Gamma}^{\text{prf}}(B)$  denotes distinguishing advantage between  $\Gamma_K$  for uniform  $K$  and a random function into  $\mathbb{F}^\ell$ . For  $H_{\text{key}}$ ,  $\text{Adv}_{H_{\text{key}}}^{\text{kdf}}(B)$  denotes the advantage in distinguishing  $H_{\text{key}}(k_P, k_V, \text{ctx})$  from a uniform PRF key when at least one input share is uniform and hidden until both key-share commitments are fixed. For the malicious-secure 2PC protocol realizing the generic digest circuits,  $\text{Adv}_{\text{HDig}}^{2\text{pc}}(B)$  denotes the real/ideal simulation advantage, including malicious abort behavior and output delivery. For the cached encrypted-share generation protocol,  $\text{Adv}_{\text{tagshare-gen}}^{\text{tagshare-gen}}(B)$  denotes the real/ideal simulation and soundness advantage of the  $\text{TagShareGen}$  circuit. For the aggregate decryption/equality protocol,  $\text{Adv}_{\text{shareagg}}^{\text{shareagg}}(B)$  denotes the real/ideal simulation advantage of  $\Pi_{\text{ShareAgg}}$ , including malicious abort behavior and output delivery. For the session aggregate-binding relation,  $\text{Adv}^{\text{aggbind}}(B)$  denotes the probability that  $R_{\text{ShareAggBind}}$  accepts while the active owner-share aggregate, active encrypted verifier-share aggregate, epoch owner-share aggregate, or epoch encrypted verifier-share aggregate is not the canonical value determined by the verified authenticated share handles and the accepted epoch digest handle. For the public-key encryption layer,  $\text{Adv}^{\text{pke}}(B)$  denotes the sum of semantic-security, homomorphic-correctness, and decryption-soundness advantages needed for encrypted verifier shares. For the share-authentication mechanism,  $\text{Adv}_{\text{authshare}}^{\text{authshare}}(B)$  denotes the probability of producing an accepting authenticated share tuple  $(\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, \text{role}, \cdot, \mu)$  that was not output by  $\text{TagShareGen}$  for that tuple.

For the proof system,  $\text{Adv}_{\text{arith}}^{\text{ks}}(B)$  is the probability that an accepting arithmetic commit-and-prove transcript has no extractor output satisfying the arithmetic relation.  $\text{Adv}_{\text{pair}}^{\text{snd}}(B)$  is the probability that the pairing-product proof accepts while the linked group elements do not satisfy the stated pairing equations.  $\text{Adv}_{\text{link}}^{\text{bind}}(B)$  is the probability that two different scalar representatives or exponent handles are accepted as linked to the same scalar-link commitment.  $\text{Adv}_{\text{H}_{\text{elt}}}^{\text{coll}}(B)$  is the probability of a representative collision  $\text{H}_{\text{elt}}(x) = \text{H}_{\text{elt}}(x')$  for distinct extracted elements.

The BPEB advantage is

$$\text{Adv}_N^{\text{BPEB}}(B) = \Pr \left[ Q \neq 0, \deg Q \leq N, g_1^{Q(\tau)} = 1 : Q \leftarrow B(g_1, g_1^\tau, \dots, g_1^{\tau^N}) \right].$$

This is the bounded-degree polynomial binding property used by KZG-style polynomial commitments. In the algebraic group model, it is implied by the usual KZG binding/ $q$ -SDH-style assumption for  $q \geq N$ , up to the standard reduction loss.

**Lemma 5 (BPEB from KZG).** *Assume the SRS  $(g_1, g_1^\tau, \dots, g_1^{\tau^q})$  is generated for the KZG binding game and  $N \leq q$ . Suppose  $B$  wins the BPEB game by*

outputting a nonzero polynomial

$$Q(X) = \sum_{i=0}^N q_i X^i \quad \text{with} \quad g_1^{Q(\tau)} = 1.$$

Then there is an algebraic adversary  $B'$  that wins the bounded-degree KZG binding game with the same success probability, up to the standard extraction loss of the algebraic group model.

*Proof.* Adversary  $B'$  receives the KZG SRS and forwards it to  $B$ . When  $B$  outputs the coefficient vector of  $Q$ ,  $B'$  interprets it as two distinct degree- $N$  polynomial openings to the same KZG commitment: the zero polynomial and  $Q$ . The commitment to the zero polynomial is  $1_{G_1}$ , while the commitment to  $Q$  is  $g_1^{Q(\tau)}$ , which equals  $1_{G_1}$  by the BPEB winning condition. Since  $Q \neq 0$  and  $\deg Q \leq N$ , these are two different bounded-degree polynomials with the same KZG commitment, violating polynomial binding. In the algebraic group model, the coefficient vector extracted from  $B$ 's algebraic representation gives exactly the polynomial needed by the binding adversary. Therefore the BPEB success probability is bounded by the bounded-degree KZG binding, and hence by the corresponding  $q$ -SDH-style binding assumption for  $q \geq N$ .

## B.2 Owner-hidden digest security

The digest proof is the only place where adaptive rejected equality tests matter. The proof first idealizes key derivation, 2PC, and the PRF, and then analyzes the remaining information-theoretic experiment over random tags.

*Digest exactness experiment.* The challenger runs `DigKeyReg` and maintains an extracted certified set  $S$  and digest shares for  $A(S)$ . The adversary adaptively submits session attempts. Verification is ordered as syntax, no-injection, same-element binding, digest 2PC, and PSI. The counter  $q_{\text{dig}}$  is incremented exactly for attempts reaching the digest 2PC. The adversary wins if one such attempt accepts the digest check with an extracted strict subset  $R \subsetneq S$ .

*Game  $G_0$ : real execution.* This is the real digest execution with real key-share commitments, real KDF output, real 2PC transcripts, and real PRF evaluations.

*Game  $G_1$ : uniform digest key.* Replace  $K_{\text{dig}} = H_{\text{key}}(k_{\text{dig}}^P, k_{\text{dig}}^V, \text{ctx})$  by an independent uniform key. Any distinguisher between  $G_0$  and  $G_1$  yields a KDF-unpredictability adversary, unless a key-share commitment is opened inconsistently or the honest share is distinguished before opening. Thus

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \text{Adv}_{\text{Com}}^{\text{hide}}.$$

*Game  $G_2$ : ideal digest circuits.* Replace each malicious-secure 2PC execution of `DigSetup`, `DigUpdate`, and `DigSession` by its ideal circuit functionality and simulator. The ideal functionality checks the same public commitments and outputs only the specified shares, public commitments, abort stage, and final equality bit. Therefore

$$|\Pr[G_1 = 1] - \Pr[G_2 = 1]| \leq \text{Adv}_{\text{HDig}}^{2\text{pc}}.$$

Binding of  $D_t^P, D_t^V$  is included in the event that a party supplies shares not opening to the public epoch commitments.

*Game  $G_3$ : random-function tags.* Replace  $\Gamma_{K_{\text{dig}}}(\text{digest} \parallel \text{applID} \parallel \text{partyID} \parallel \text{statelD} \parallel x)$  by a random function  $R(x)$  into  $\mathbb{F}^\ell$  on the stable digest domain. Then

$$|\Pr[G_2 = 1] - \Pr[G_3 = 1]| \leq \text{Adv}_F^{\text{prf}}.$$

*Adaptive rejection analysis in  $G_3$ .* Let  $Q = |\mathbb{F}^\ell|$  and let  $V$  be the full tag space for the extracted set  $S$ . Fix an attempt  $i$  that reaches the digest circuit, and condition on the adversary's entire prior view  $V_{i-1}$ , including all previous abort stages and digest equality bits. The verification order ensures that the candidate active set  $R_i$  is fixed before the  $i$ -th digest equality bit is returned. If  $R_i = S$ , the strict-omission bad event does not occur. Otherwise let  $O_i = S \setminus R_i \neq \emptyset$ . Acceptance is equivalent to

$$\sum_{x \in O_i} R(x) = 0.$$

For every nonempty omitted set  $O \subseteq S$ , define the zero-sum subspace

$$H_O = \left\{ (R(x))_{x \in S} \in V : \sum_{x \in O} R(x) = 0 \right\}.$$

This subspace has relative density  $1/Q$ . A previous rejected digest check reveals only that the tag vector is outside the corresponding subspace  $H_{O_j}$ ; it does not reveal the value of the sum, any coordinate of a tag, or any other linear predicate. Thus, before the  $i$ -th equality test and conditioned on no prior accepting strict omission, the tag vector is uniform over a subset obtained from  $V$  by deleting at most  $i - 1$  such subspaces. By the union bound this conditioned sample space has size at least

$$|V| \left( 1 - \frac{i-1}{Q} \right).$$

The new accepting event is contained in one subspace of size  $|V|/Q$ . Let  $E_i$  denote the event that attempt  $i$  is an accepting strict omission. Hence

$$\Pr[E_i \mid V_{i-1}] \leq \frac{|V|/Q}{|V|(1 - (i-1)/Q)} = \frac{1}{Q - (i-1)}.$$

Summing over the at most  $q_{\text{dig}}$  attempts that reach the digest circuit gives

$$\Pr[G_3 = 1] \leq \sum_{i=1}^{q_{\text{dig}}} \frac{1}{Q - (i - 1)} \leq \frac{q_{\text{dig}}}{Q - q_{\text{dig}}}.$$

The condition  $q_{\text{dig}} < Q$  is explicit. If  $q_{\text{dig}}$  is polynomial and  $Q = |\mathbb{F}|^\ell$  is superpolynomial, then this is  $q_{\text{dig}}/Q + \text{negl}(\lambda)$ ; if  $q_{\text{dig}} \leq Q/2$ , it is at most  $2q_{\text{dig}}/Q$ .

Combining the games proves

$$\text{Adv}^{\text{dig-exact}} \leq \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_{\text{HDig}}^{2\text{pc}} + \text{Adv}_\Gamma^{\text{prf}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \text{Adv}_{\text{Com}}^{\text{hide}} + \frac{q_{\text{dig}}}{|\mathbb{F}|^\ell - q_{\text{dig}}}.$$

### B.3 Authenticated encrypted-share generation proof

This subsection proves Lemma 2. The proof is a concrete game-based reduction for the encrypted-share version of **TagShareGen**. The adversary may corrupt either party in polynomially many executions and may issue share-verification queries. It wins if it outputs either: (i) an accepting owner/encrypted-verifier handle for a commitment  $C_x$  that is not table-generated for the extracted element; (ii) an accepting handle whose decrypted verifier share and owner share do not sum to the stable digest tag of the extracted element; or (iii) a privacy bit distinguishing a real corrupted-party view from the simulated view specified below. Let  $\text{Win}_{G_i}$  be the corresponding event in Game  $G_i$ .

*Game  $G_0$ : real execution.* The challenger runs real key-share commitments, the real **TagShareGen** 2PC protocol, the real public-key encryption algorithm, the real PRF  $\Gamma_{K_{\text{dig}}}$ , and the real share-authentication algorithm. It answers **VerifyShare** and **VerifyEncShare** queries using the real predicates. Thus

$$\Pr[\text{Win}_{G_0}] = \Pr[\text{Exp}_A^{\text{tagshare}} = 1].$$

*Game  $G_1$ : key-derivation hybrid.* Game  $G_1$  replaces

$$K_{\text{dig}} = H_{\text{key}}(k_{\text{dig}}^P, k_{\text{dig}}^V, \text{appID}, \text{partyID}, \text{stateID})$$

by an independent uniform key after both key-share commitments are fixed. If  $\mathcal{A}$  distinguishes  $G_0$  from  $G_1$ , a KDF adversary embeds its challenge key in the protocol. Inconsistent key-share openings give a commitment-binding adversary, and distinguishing the honest key share before opening gives a commitment-hiding adversary. Therefore

$$|\Pr[\text{Win}_{G_0}] - \Pr[\text{Win}_{G_1}]| \leq \text{Adv}_{H_{\text{key}}}^{\text{kdf}}(B_1) + \text{Adv}_{\text{Com}}^{\text{bind}}(B'_1) + \text{Adv}_{\text{Com}}^{\text{hide}}(B''_1).$$

*Game  $G_2$ : ideal encrypted-share generation.* Game  $G_2$  replaces each real 2PC execution by the ideal functionality  $\mathcal{F}_{\text{TagShareGen}}$  for the circuit in Section 7. On public input  $(\text{tagCtx}, \text{ck}_{\text{dig}}, C_K^P, C_K^V, pk_V, vk_{\text{share}}, C_x)$  and private inputs  $(x, r_x^C, k_{\text{dig}}^P, \rho_K^P)$  from  $P$  and  $(k_{\text{dig}}^V, \rho_K^V, sk_{\text{share}})$  from  $V$ , the functionality verifies

$$C_x = \text{Com}_X(x; r_x^C), \quad C_K^P = \text{Com}(k_{\text{dig}}^P; \rho_K^P), \quad C_K^V = \text{Com}(k_{\text{dig}}^V; \rho_K^V).$$

It computes the stable tag

$$\gamma_x = \Gamma_K(\text{digest} \parallel \text{applD} \parallel \text{partyID} \parallel \text{stateID} \parallel x),$$

samples  $s_x^P \leftarrow \mathbb{F}^\ell$ , sets  $s_x^V = \gamma_x - s_x^P$ , computes

$$E_x^V = \text{Enc}_{pk_V}(s_x^V; \omega_x),$$

and creates the two authenticated tuples

$$(\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, P, s_x^P, E_x^V, \mu_x^P), \quad (\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, V, E_x^V, \mu_x^V).$$

It outputs only

$$(\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, s_x^P, E_x^V, \mu_x^P)$$

to  $P$  and only

$$(\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, E_x^V, \mu_x^V)$$

to  $V$ . By malicious 2PC security for this circuit, the replacement changes the winning probability by at most

$$|\Pr[\text{Win}_{G_1}] - \Pr[\text{Win}_{G_2}]| \leq \text{Adv}^{\text{tagshare-gen}}(B_2).$$

*Game  $G_3$ : authenticated-handle table.* The challenger stores every tuple output by  $\mathcal{F}_{\text{TagShareGen}}$  in a table  $\mathcal{T}$ . Verification accepts only table entries. If  $G_2$  and  $G_3$  differ,  $\mathcal{A}$  has produced a fresh authenticated tuple of one of the forms

$$(\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, P, s, E, \mu) \quad \text{or} \quad (\text{tagCtx}, \text{ck}_{\text{dig}}, C_x, V, E, \mu)$$

that verifies but was not output by the ideal functionality. This is an authentication forgery, so

$$|\Pr[\text{Win}_{G_2}] - \Pr[\text{Win}_{G_3}]| \leq \text{Adv}^{\text{authshare}}(B_3).$$

*Game  $G_4$ : encryption privacy hybrid.* For privacy against a malicious owner, replace every ciphertext  $E_x^V = \text{Enc}_{pk_V}(s_x^V; \omega_x)$  in the owner's output by an encryption of  $0^\ell$  while keeping the ideal table for soundness. A distinguisher yields an IND-CPA adversary for the encryption scheme. Correctness/decryption-soundness failures of the encryption scheme are also charged to  $\text{Adv}^{\text{pke}}$ . Thus

$$|\Pr[\text{Win}_{G_3}] - \Pr[\text{Win}_{G_4}]| \leq \text{Adv}^{\text{pke}}(B_4).$$

This game is used only for privacy. For soundness of decrypted shares, the table in  $G_3$  records the plaintext  $s_x^V$  associated with each ciphertext and the decryption-correctness loss has already been charged to  $\text{Adv}^{\text{pke}}$ .

*Game  $G_5$ : random-function tags.* Replace  $\Gamma_K$  on the stable digest domain by a random function  $R : U \rightarrow \mathbb{F}^\ell$ . Any distinguisher gives a PRF adversary, so

$$|\Pr[\text{Win}_{G_4}] - \Pr[\text{Win}_{G_5}]| \leq \text{Adv}_F^{\text{prf}}(B_5).$$

*Analysis of  $G_5$ .* For every table entry associated with an extracted element  $x$ , the ideal functionality samples  $s_x^P$  uniformly, defines  $s_x^V = R(x) - s_x^P$ , and records a ciphertext that decrypts to  $s_x^V$  except with encryption-correctness loss. Hence

$$s_x^P + \text{Dec}_{sk_V}(E_x^V) = R(x)$$

for every table-generated owner/encrypted-verifier pair. The owner's marginal view contains  $s_x^P$ , which is uniform, and an encryption independent of the plaintext in the privacy hybrid. Thus for all  $a, t \in \mathbb{F}^\ell$ ,

$$\Pr[s_x^P = a \mid R(x) = t] = |\mathbb{F}|^{-\ell},$$

and the owner learns neither  $R(x)$  nor the verifier share. The verifier's view is simulated without  $x$  because the only use of  $x$  occurs inside the ideal functionality and the 2PC simulator has already hidden the owner's input.

For soundness, every accepting handle in  $G_5$  appears in  $\mathcal{T}$ . If a commitment  $C_x$  is associated with two different extracted elements, the adversary breaks binding of  $\text{Com}_X$ . Otherwise the table fixes a unique extracted element and the generated handles satisfy the encrypted-share relation above. Therefore

$$\Pr[\text{Win}_{G_5}] \leq \text{Adv}_{\text{Com}_X}^{\text{bind}}(B_6),$$

where privacy distinguishing has been accounted for in the previous hybrids. Summing all game transitions proves Lemma 2.

#### B.4 Cached encrypted-share digest realization

This subsection proves Lemma 3. The adversary controls the state owner and may adaptively submit updates and sessions. A digest attempt is counted in  $q_{\text{dig}}$  only when syntax, no-injection, same-element binding, share-handle verification, the aggregate-binding relation  $R_{\text{ShareAggBind}}$ , digest-share commitment-opening checks, and the aggregate decryption/equality 2PC  $\Pi_{\text{ShareAgg}}$  are reached.

*Bad events.* Let  $\text{Bad}_{\text{gen}}$  be the event that an accepted owner share or encrypted verifier-share handle was not produced by  $\text{TagShareGen}$  for the element commitment named in the authenticated tuple. Let  $\text{Bad}_{\text{auth}}$  be the event that a handle verifies although it does not appear in the ideal table of Appendix B.3. Let  $\text{Bad}_{\text{link}}$  be the event that same-element binding accepts while the owner share, encrypted verifier-share handle, PSI token, accumulator representative, and extracted element commitment are not all linked to the same element. Let  $\text{Bad}_{\text{aggbind}}$  be the event that  $R_{\text{ShareAggBind}}$  accepts but one of the values later supplied to  $\Pi_{\text{ShareAgg}}$  is not the canonical aggregate certified by the relation: either

$$A_{\text{use}}^P \neq \sum_{x \in S_{\text{use}}} s_x^P, \quad E_{\text{use}}^V \neq \bigoplus_{x \in S_{\text{use}}} E_x^V,$$

or the epoch values  $(A_t^P, E_t^V)$  are not the authenticated aggregates bound to the accepted epoch handle. Let  $\text{Bad}_{\text{agg}}$  be the event that  $\Pi_{\text{ShareAgg}}$  accepts or leaks

a view inconsistent with its ideal aggregate-equality functionality on the public inputs certified by  $R_{\text{ShareAggBind}}$ . Finally, let  $\text{Bad}_{\text{omit}}$  be the event that no previous bad event occurs and an accepted session uses a strict subset of the extracted state.

By Lemma 2, authentication unforgeability, encryption security/correctness, and aggregate-check 2PC security, let  $\text{Bad}_{\text{sharetag}} = \text{Bad}_{\text{gen}} \vee \text{Bad}_{\text{auth}} \vee \text{Bad}_{\text{aggbind}} \vee \text{Bad}_{\text{agg}}$ . Then

$$\begin{aligned} \Pr[\text{Bad}_{\text{sharetag}}] \leq & \text{Adv}^{\text{tagshare-gen}} + \text{Adv}^{\text{shareagg}} + \text{Adv}^{\text{aggbind}} + \text{Adv}^{\text{authshare}} \\ & + \text{Adv}^{\text{pke}} + \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_{\Gamma}^{\text{prf}} + \text{Adv}_{\text{Com}}^{\text{bind/hide}}. \end{aligned}$$

The event  $\text{Bad}_{\text{link}}$  is bounded by soundness of same-element binding and link commitments. The event  $\text{Bad}_{\text{aggbind}}$  is bounded by soundness of  $R_{\text{ShareAggBind}}$ , binding of  $D_t^P$  and of the 2PC input-binding commitments  $B_{\text{use}}^P, B_t^P$ , authentication of the encrypted-share handles, and homomorphic-correctness of the encryption scheme. More explicitly, if  $R_{\text{ShareAggBind}}$  accepts with a noncanonical active aggregate, then either a share handle verifies without appearing in the authenticated table, the proof system accepts a false aggregate equation, or the adversary opens one of  $D_t^P, B_{\text{use}}^P, B_t^P$  inconsistently. These cases are bounded by  $\text{Adv}^{\text{aggbind}} + \text{Adv}^{\text{authshare}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \text{Adv}^{\text{pke}}$ .

*Conditioning on no bad event.* Condition on the complement of all bad events except  $\text{Bad}_{\text{omit}}$ . Because  $R_{\text{ShareAggBind}}$  and the input-binding checks in  $\Pi_{\text{ShareAgg}}$  both accept, the public inputs to the aggregate-equality 2PC are the same canonical aggregates certified by the session relation. For every accepted epoch  $t$ ,

$$A_t^P = \sum_{x \in S_t} s_x^P, \quad \text{Dec}_{sk_V}(E_t^V) = \sum_{x \in S_t} s_x^V,$$

where each generated handle satisfies

$$s_x^P + s_x^V = \Gamma_{K_{\text{dig}}}(\text{digest} \parallel \text{applID} \parallel \text{partyID} \parallel \text{statelD} \parallel x)$$

for the same element  $x$  extracted by no-injection and same-element binding. Hence

$$A_t^P + \text{Dec}_{sk_V}(E_t^V) = \sum_{x \in S_t} \Gamma_{K_{\text{dig}}}(x).$$

For any accepted session with active set  $R$ ,  $\Pi_{\text{ShareAgg}}$  accepts only if

$$A_{\text{use}}^P + \text{Dec}_{sk_V}(E_{\text{use}}^V) = A_t^P + \text{Dec}_{sk_V}(E_t^V),$$

so

$$\sum_{x \in R} \Gamma_{K_{\text{dig}}}(x) = \sum_{x \in S_t} \Gamma_{K_{\text{dig}}}(x).$$

The malicious owner cannot evaluate this equality offline. It has only random owner shares and ciphertexts of verifier shares. By encryption privacy and the fact that  $\Pi_{\text{ShareAgg}}$  releases only an aggregate equality bit, any effective full-digest equality attempt must invoke  $\Pi_{\text{ShareAgg}}$  and is counted in  $q_{\text{dig}}$ .

*Strict omission.* If  $\text{Bad}_{\text{omit}}$  occurs, no-injection gives  $R \subsetneq S_t$ . Let  $O = S_t \setminus R$ , which is nonempty. The accepted digest equality implies

$$\sum_{x \in O} \Gamma_{K_{\text{dig}}}(x) = 0.$$

Applying the adaptive hidden-digest rejection analysis of Appendix B.2 to the protocol-visible equality attempts gives

$$\Pr[\text{Bad}_{\text{omit}}] \leq \frac{q_{\text{dig}}}{|\mathbb{F}|^\ell - q_{\text{dig}}}$$

plus the PRF and KDF losses already included above.

*Conclusion.* Taking a union bound over the bad events yields

$$\begin{aligned} \text{Adv}_{\Pi_{\text{ShareTag}}}^{\text{hdig}} &\leq \text{Adv}^{\text{tagshare-gen}} + \text{Adv}^{\text{shareagg}} + \text{Adv}^{\text{aggbind}} + \text{Adv}^{\text{authshare}} \\ &\quad + \text{Adv}^{\text{pke}} + \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_{\Gamma}^{\text{prf}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \frac{q_{\text{dig}}}{|\mathbb{F}|^\ell - q_{\text{dig}}}. \end{aligned}$$

This proves Lemma 3.

## B.5 Exact-PICS compiler proof

Consider  $\text{Exp}_{\text{ExactPICS}, A}^{\text{exact}}$ . Let  $\text{Bad}_{\text{setup}}$  be the event that the base setup accepts but the extractor fails to output a canonical set  $S$ . Let  $\text{Bad}_{\text{noinj}}$  be the event that a session accepts with extracted  $S_{\text{use}} \not\subseteq S$ . Let  $\text{Bad}_{\text{sebind}}$  be the event that a session accepts although token, commitment, representative, and digest inputs are not the same hidden element. Let  $\text{Bad}_{\text{dig}}$  be the event that no previous bad event occurs and an accepted session has  $S_{\text{use}} \subsetneq S$ .

If the adversary wins, then one of these four bad events occurs. Indeed, if setup extraction succeeds and no-injection and same-element binding hold, every accepted session has a distinct extracted set  $S_{\text{use}} \subseteq S$  wired to the digest inputs. If  $S_{\text{use}} \neq S$ , it is a strict subset and digest acceptance is precisely  $\text{Bad}_{\text{dig}}$ . Therefore

$$\Pr[\text{Exp}_{\text{ExactPICS}, A}^{\text{exact}} = 1] \leq \Pr[\text{Bad}_{\text{setup}}] + \Pr[\text{Bad}_{\text{noinj}}] + \Pr[\text{Bad}_{\text{sebind}}] + \Pr[\text{Bad}_{\text{dig}}].$$

The first three probabilities are bounded by the corresponding setup-extraction, no-injection, and same-element-binding advantages of the compatible backend. The fourth is bounded by the owner-hidden digest advantage  $\text{Adv}_{\Pi}^{\text{hdig}}$ . This proves the modular theorem. Substituting Lemma 1 or Lemma 3 gives the corresponding concrete bound.

## B.6 Static backend realizability

The static bilinear-accumulator backend is a realizability backend for the compatible no-injection committed-token PSI interface; it is not a performance-optimized PSI construction. Setup extraction follows from the arithmetic extractor for the



committed representatives and the accumulator set polynomial. For no-injection, suppose an accepting active record extracts to  $x \notin S$ . The arithmetic layer extracts  $a_x = H_{\text{elt}}(x)$  and the link proof binds this representative to the token and digest input. The pairing layer accepts a membership equation for  $a_x$  under the accumulator for  $S$ . Unless  $a_x$  collides with a representative in  $S$ , this is a false membership statement and contradicts pairing-equation soundness or accumulator binding. Distinctness is extracted from the arithmetic proof. Token privacy is delegated to the committed-token PSI functionality. Hence the backend realizes the interface with loss

$$\text{Adv}_{\text{arith}}^{\text{ks}} + \text{Adv}_{\text{pair}}^{\text{snd}} + \text{Adv}_{\text{link}}^{\text{bind}} + \text{Adv}_{H_{\text{elt}}}^{\text{coll}} + \text{Adv}^{\text{tok-priv}}.$$

### B.7 Bilinear HDUA setup and update soundness

The dynamic soundness proof is an induction over epochs. The induction invariant is that every accepted public epoch has a unique extracted set polynomial and accumulator value. The update proof then shows that the next extracted set is exactly the canonical set update.

We prove by induction over accepted epochs. In the base case, an accepting setup proof yields, by arithmetic knowledge soundness, extracted elements  $S_0$ , representatives  $a_x = H_{\text{elt}}(x)$ , a polynomial  $P_{S_0}(X) = \prod_{x \in S_0} (X - a_x)$ , a raw accumulator  $A_0^{\text{Acc}}$ , and an opening of  $C_0^{\text{Acc}}$ . Pairing-equation soundness and link soundness imply  $A_0^{\text{Acc}} = g_1^{P_{S_0}(\tau)}$ . If another set  $S'_0$  explained the same public commitment, commitment binding gives the same raw accumulator. The extractor obtains  $P_{S'_0}$ , and BPEB is violated by  $Q = P_{S_0} - P_{S'_0}$  unless  $S_0 = S'_0$  or a representative collision occurs. Thus the initial state is unique except for the stated losses.

For the induction step, assume that the latest accepted state at epoch  $t-1$  is uniquely extracted as  $S_{t-1}$  with accumulator  $g_1^{P_{S_{t-1}}(\tau)}$ . From an accepting update proof, the update extractor obtains insertion and deletion sets  $I_t, D_t$ , representatives, polynomials  $P_{I_t}, P_{D_t}$ , group elements  $G_I, G_D$ , and the new raw accumulator  $A_t^{\text{Acc}}$ . Membership proofs for deletions imply  $D_t \subseteq S_{t-1}$  except under the no-false-membership loss. Nonmembership proofs for insertions imply  $I_t \cap S_{t-1} = \emptyset$  except under the no-false-nonmembership loss. Distinctness and disjointness are arithmetic extracted. The batch-polynomial relation gives

$$G_I = g_2^{P_{I_t}(\tau)}, \quad G_D = g_2^{P_{D_t}(\tau)},$$

with coefficients extracted from the same representatives used in membership, nonmembership, digest update, and same-element binding. Pairing soundness gives

$$e(g_1^{P_{S_{t-1}}(\tau)}, g_2^{P_{I_t}(\tau)}) = e(A_t^{\text{Acc}}, g_2^{P_{D_t}(\tau)}).$$

Let  $S_t^* = (S_{t-1} \cup I_t) \setminus D_t$ . Canonical update validity gives the polynomial identity

$$P_{S_t^*}(X)P_{D_t}(X) = P_{S_{t-1}}(X)P_{I_t}(X).$$

By nondegeneracy of the pairing and the previous equality,  $A_t^{\text{Acc}} = g_1^{P_{S_t^*}(\tau)}$  except under pairing soundness failure. If the public new commitment also opened to a different state polynomial  $P_{S_t}$ , the BPEB reduction above applied to  $P_{S_t} - P_{S_t^*}$  gives a contradiction. Stateful verification ensures that only updates from the verifier's stored latest handle are accepted. This completes the induction and proves the theorem's bound.

### B.8 Digest-native session exactness

Fix an accepted session at epoch  $t$ . If setup/update soundness fails, charge  $\text{Adv}_{\text{biHDUA}}^{\text{upd}}$ . Otherwise the epoch has a unique extracted state  $S_t$ . If session no-injection fails, charge  $\text{Adv}_{\text{sess}}^{\text{noinj}}$ . If same-element binding or distinctness fails, charge the corresponding proof/link advantages. Conditioned on none of these events, the active records extract to a set  $S_{\text{use}} \subseteq S_t$  and the digest inputs are exactly those records. If the accepted session is not exact, then  $S_{\text{use}} \subsetneq S_t$  and  $\text{DigSession}$  has accepted a strict omitted subset. the owner-hidden digest advantage  $\text{Adv}_{\text{H}}^{\text{hdig}}$  bounds that event. Summing the bad events proves the modular session exactness theorem; substituting either digest realization gives the corresponding concrete bound.

### B.9 Hidden-state privacy

The privacy proof is independent of the soundness extractor. In privacy hybrids, proofs are simulated and commitments are replaced by hiding commitments to dummy values. Therefore the simulator does not need to know the hidden elements when simulating equal-leakage histories.

Let  $\mathcal{H}_0, \mathcal{H}_1$  be equal-leakage histories. In  $H_0$  the challenger runs the real protocol on branch  $b$ . In  $H_1$ , all arithmetic and pairing proofs are replaced by simulated zero-knowledge proofs. The distinguishing gap is bounded by the ZK advantages of the proof layers. In  $H_2$ , accumulator commitments and digest public handles are replaced by dummy handles of the same types. No equivocation is required because the proofs are already simulated; hiding of the commitments bounds the transition. In  $H_3$ , all digest 2PC transcripts are replaced by the simulator guaranteed by malicious-secure 2PC. The simulator uses only sizes, public abort stages, output delivery information, and the final digest accept/reject bit, all of which are included in the leakage. In  $H_4$ , committed-token and flagged-PSI transcripts are replaced by their simulators using session identifiers, active-record counts, abort stages, and authorized outputs. At this point the transcript consists only of simulated proofs, dummy commitments, simulated 2PC/PSI messages, and explicit leakage. Since the two histories have identical leakage, the final distributions are identical. Summing hybrid losses gives

$$\text{Adv}^{\text{priv}} \leq \text{Adv}_{\text{arith}}^{\text{zk}} + \text{Adv}_{\text{pair}}^{\text{zk}} + \text{Adv}_{\text{Com}}^{\text{hide}} + \text{Adv}_{\text{HDig}}^{\text{2pc-priv}} + \text{Adv}^{\text{tok-priv}} + \text{Adv}^{\text{PSI-sim}}.$$

### B.10 VA-UPSI composition

*Ideal functionality.* The functionality  $\mathcal{F}_{VA-UPSI}^{\text{exact}}$  stores for each party  $P_i$  a state  $S_{i,t}$  and latest handle  $h_{i,t}$ . It supports  $\text{Init}(P_i, S_{i,0})$ , canonical  $\text{Update}(P_i, \Delta^+, \Delta^-)$ ,  $\text{Query}(\text{sid}, t_X, t_Y, f)$  that returns only the authorized value  $f(X_{t_X} \cap Y_{t_Y})$  or its prescribed shares when both state handles are current, and  $\text{Abort}(\text{stage}, \text{role})$ . The baseline theorem is for static malicious corruption of at most one party. The leakage consists of state sizes, update sizes and directions, session active-record counts, public abort stage and role, and authorized output type.

We give the simulator for static corruption of one party. If the state owner is corrupted, the simulator runs the real verification algorithms and uses the setup, update, and session extractors. Every accepted update extends the unique extracted state by Appendix B.7. For a query, the simulator extracts  $S_{\text{use}}$  from the session proof and uses Appendix B.8 to conclude  $S_{\text{use}} = S_t$  except with the stated exactness loss. It sends  $S_t$  to the ideal functionality and invokes the flagged-PSI simulator on the authorized output. All non-output messages are simulated by the ZK, commitment-hiding, digest-2PC, and token-privacy simulators.

If the verifying party is corrupted and the state owner is honest, the key-share registration prevents unilateral bias of the digest key when at least one share is honest. The simulator uses the privacy hybrids of Appendix B.9 to replace the honest party's hidden accumulators, digest state, update elements, and tokens by simulated objects consistent with leakage. Selective aborts by the corrupted verifier are reflected in the ideal functionality through  $\text{Abort}(\text{stage}, \text{role})$ . Thus the real and ideal executions are indistinguishable up to the sum of the extraction, exactness, privacy, digest-realization, proof-system, token, PSI, and key-derivation losses.

Combining all subproofs, a compact modular master bound is

$$\begin{aligned} \text{Adv}_A^{\text{VA-UPSI}} \leq & \text{Adv}_\Pi^{\text{hdig}} + \text{Adv}_{\text{arith}}^{\text{ks}} + \text{Adv}_{\text{pair}}^{\text{snd}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \text{Adv}_{\text{Com}}^{\text{hide}} + \text{Adv}_{\text{link}}^{\text{bind}} \\ & + \text{Adv}^{\text{BPEB}} + \text{Adv}_{\text{H}_{\text{elt}}}^{\text{coll}} + \text{Adv}^{\text{tok-priv}} + \text{Adv}^{\text{PSI-sim}}. \end{aligned}$$

For  $\Pi = \Pi_{\text{HDig}}^{2\text{PC}}$ , substitute

$$\text{Adv}_\Pi^{\text{hdig}} \leq \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_{\text{HDig}}^{2\text{pc}} + \text{Adv}_\Gamma^{\text{prf}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \frac{q_{\text{dig}}}{|\mathbb{F}|^\ell - q_{\text{dig}}}.$$

For  $\Pi = \Pi_{\text{HDig}}^{\text{ShareTag}}$ , substitute

$$\begin{aligned} \text{Adv}_\Pi^{\text{hdig}} \leq & \text{Adv}^{\text{tagshare-gen}} + \text{Adv}^{\text{shareagg}} + \text{Adv}^{\text{aggbind}} + \text{Adv}^{\text{authshare}} + \text{Adv}^{\text{pke}} \\ & + \text{Adv}_{H_{\text{key}}}^{\text{kdf}} + \text{Adv}_\Gamma^{\text{prf}} + \text{Adv}_{\text{Com}}^{\text{bind}} + \frac{q_{\text{dig}}}{|\mathbb{F}|^\ell - q_{\text{dig}}}. \end{aligned}$$